

EVALUATING LEXICAL, EMBEDDING, AND TRANSFORMER-BASED
MODELS UNDER QUALITY-EFFICIENCY TRADEOFFS FOR MUSIC
PLAYLIST RECOMMENDATION

By

JOSE SANTIAGO CAMPA MORALES

A Thesis Submitted to The W.A. Franke Honors College

In Partial Fulfillment of the bachelor's degree
With Honors in

Computer Science

THE UNIVERSITY OF ARIZONA

M A Y 2 0 2 6

Approved by:

Dr. Eduardo Blanco
Department of Computer Science

Evaluating Lexical, Embedding, and Transformer-Based Models Under Quality-Efficiency Tradeoffs for Music Playlist Recommendation

Jose Santiago Campa Morales

Abstract

Music playlist recommendation is a core language task of music streaming services, requiring models to balance semantic similarity between tracks with playlist-specific contexts. A challenge in this domain is that playlist titles are inherently ambiguous and need not align with their contents, making accurate recommendation difficult. This study evaluates eleven recommendation baselines on the Spotify Playlists dataset, with seven single recommenders including naive models (random and popularity), lexical models (TF-IDF and BM25), Word2Vec embeddings, transformer-based embeddings (both unmodified and parameter-efficient fine-tuned via low-rank adaptation), and four hybrid combinations with two different weight sets each. Experiments were conducted on over 156,000 playlists and 2.7 million tracks, with playlists being capped at 500 songs to mitigate outliers. Performance was measured using both unranked metrics (Precision, Recall, F1 Score, Hit Rate) and ranked metrics (MRR, MAP, NDCG). Results demonstrate that hybrid models achieved the strongest performance, particularly those combining lexical baselines, achieving over 60% hit rate. In contrast, parameter-efficient fine-tuned transformer models provided marginal gains at a substantially higher computational cost. These findings establish a benchmark for evaluating quality-efficiency tradeoffs in music playlist recommendation and provide a foundation for future conversational recommender systems, especially those in resource-constrained environments.

1 Introduction & Motivation

Music recommendation has become an important task in music streaming services by providing a personalized experience to its users. This personalization not only introduces users to new tracks and artists, but a strong recommender system maintains a user active within the platform.

A common approach to music recommendation is automatic playlist generation. Music streaming services hold millions of tracks and artists catered for each individual user. To organize these tracks and promote user interactivity, these platforms automatically create playlists based on a common trend or subject such as popularity, region or genre. These playlists function as initial user exposure to new artists based on their preferences. Additionally, users can create and tailor their own individualized playlists to sort their own music catalogue.

Playlist-driven music recommendation is challenging for various reasons. Firstly, a playlist title must semantically match with its contents. They would require encapsulating the overall context of many tracks in very few words or symbols. Playlist titles are inherently short as they represent

subjective concepts such as moods, experiences or events, making semantic consistency between titles and tracks difficult. For example, songs can mean different things to different people. Artists with a diverse catalogue of songs and genres are still usually boxed to only one genre. Some tracks have mismatched lyrics and melody. Automatic playlist generation excels on creating accurate statistic-based playlists such as popularity (“Top 50 Global”) or regional (“Mexican Songs”). However, context matters for more complex playlists. Common attributes between tracks can only be apparent to the user, which can negatively affect the quality of a recommender.

A more sophisticated music recommendation system involves conversation between a seeker and a recommender. Unlike regular recommendation systems, conversational recommendation systems (CRS) interact with a user while recommending items to the user based on those interactions. This study’s main focus was originally meant to cover CRS on the music domain; however, most CRS-based research is implemented on the movie domain. Dialogues based on movie recommendations exist for CRS purposes. Music recommendation lacks these datasets, which created a natural barrier between this study’s goals and its resources.

This limitation motivated converting the task to title-based playlist recommendation. By removing the conversational aspect of a recommender system, playlist titles now served as both context and user intent (goal), creating a safe alternative to conversations. Playlist titles as text input and its track list as output provided a suitable input-output dataset to run the experiments.

The experiments ranged from randomization to popularity-based. More complex experiments dealt with word embeddings and keyword identification. However, even more complex experiments used transformers as baselines. Due to their intended number of trainable parameters, these language models are expensive to train. Another motivation for this study involved the use of parameter-efficient fine-tuning (PEFT) to decrease the computational cost of training a recommendation transformer. This study focused on low rank adaptation (LoRA) as a PEFT method by reducing the number of trainable parameters in the transformer model.

The lack of a music-based CRS dataset along with the computational cost of current language models has motivated the evaluation of playlist recommendation methods. By utilizing a playlist title and tracks for playlist recommendation, this study aptly compares lexical, embedding-based and transformer-based approaches. In addition to assess recommendation quality, this study examines the tradeoff between effectiveness and computational efficiency.

2 Related Work

Recommender systems have been studied across different domains. In the music domain, the large-scale growth of music streaming services has allowed for more publicly available user statistics. Previous work in music recommendation has shown that playlist names can contain

valuable contextual information, and this listening context has provided recommenders with improved precision [1]. Other work has proposed multi-context recommenders where recommendation quality improves with the consideration of listening situations and histories and track content [2]. In addition, user-generated playlists follow music clusters, which further proves how playlist titles and content provide semantic information [3]. These papers offer playlist titles as recommendation context; however, they do not compare different retrieval-based models under a common evaluation setting.

More recently, the rise of transformers has framed conversational recommender systems as a natural language processing task, demonstrating that parameter-efficient transformer models can generalize across recommendation domains [4]. This inspired the idea of treating playlist recommendation as a language-based problem, though this study focuses on music playlists rather than movies, and does not implement a full conversational model. Conversely, studies on embedding-based recommendation have suggested that sentence-level embeddings could capture semantic relationships, surpassing state-of-the-art transformer models [5].

One of the main objectives of this study is to assess the recommendation quality of a PEFT light transformer model. To reduce the computational cost of adapting transformer models, PEFT strategies such as LoRA reduce the number of trainable parameters by freezing most pre-trained parameters [6]. Furthermore, contrastive embedding methods such as SimCSE pull semantically similar representations together and push unrelated examples [7]. While transformer models have been heavily studied and shown strong results in a recommendation environment, their quality-efficiency tradeoffs are not clearly visible in a playlist-based music recommendation compared to other lexical baselines.

3 Task & Dataset

The goal of each model is to retrieve the top 10 tracks ($K = 10$) given a playlist title. More specifically, given a playlist title query (input), each non-naive model first retrieves the 30 most similar playlists. Their songs are then used as candidates for the final recommendation. These tracks are then ranked, and the top 10 songs are returned as the final recommendation. The number of similar playlists retrieved (30) was chosen arbitrarily to balance candidate coverage with computational cost. Alternative values (e.g., 15 or 50) could affect the final rankings.

The main obstacle of this task is the input, as title playlists have shown to be of short length. Additionally, playlist titles are not entitled to match the semantic meaning of the tracks, i.e., playlist titles are ambiguous. Playlists can be created with a genre, a mood or an event in mind (or a combination of these). The semantics of tracks are also subjective, so playlist titles can misalign with the tracks.

The Spotify Playlists dataset contains user-generated playlists, with each playlist grouping songs sharing similar themes or moods, making it suitable for training and evaluating recommendation systems. Each track is composed of a unique user identifier (user who created the playlist), its track name, its artist and playlist title of the playlist it belongs to. Basic preprocessing such as cleaning quotes and whitespace was done. Given that multiple tracks can share the same title, all tracks had a “[track name] - [artist]” song field for consistency. Null markers were removed. To reduce extreme outliers, playlists were capped at 500 songs. These decisions were based on consistency and fairness, as a playlist with thousands of songs defeats the purpose of a playlist and turns into a music library.

Although the song field appropriately identifies unique tracks, duplicates were not removed. This decision was intentional, as these are user-generated playlists. Keeping the source input as intact as possible was a priority for mimicking real-world playlists. Tracks may appear with different studio, live or remastered versions, with each one being a new unique song.

After preprocessing and filtering, the dataset included 156,706 playlists and 2,755,285 songs. Across the postprocessed dataset, playlists contained a median of 20 songs per playlist, with the median song appearing once per playlist.

	min	25%	50%	75%	max	mean	std
Songs per Playlist	1	11	20	45	500	43	65.37
Playlists per Song	1	1	1	3	2606	5	21.91

Table 1: Dataset statistics for Spotify Playlists. Summary of playlist lengths and song occurrences, including minimum, quartiles, maximum, mean, and standard deviation.

For the purposes of this study, consider the following examples.

- Example #1: Mood playlist
- Playlist title: Chill out
- Playlist length: 140 songs
- Songs include:
 - Stay the Same - Mark Pritchard Remix - Bonobo
 - Suffer - Glen Porter
 - Lost In Thought - Jon Hopkins
 - Old Devil - Garage Demo - Emancipator
 - Old Devil (Kept Blue Remix) - Emancipator

This example provides an accurate playlist: a reasonable amount of semantically related tracks. While the playlist title could have more than one meaning, the songs do follow a R&B or

independent style which helps the playlist title. Also, multiple versions of the same song are in the playlist, showcasing a common occurrence in playlist creation.

- Example #2: Genre playlist
- Playlist title: HARD ROCK 2010
- Playlist length: 67 songs
- Songs include:
 - (The Angels Wanna Wear My) Red Shoes - Elvis Costello
 - I'd Rather Be With You [Radio Edit] - Joshua Radin
 - Oliver's Army - Elvis Costello & The Attractions
 - The World It Still Revolves - Garage Demo - Joe Echo
 - This Is The World We Live In - Joe Echo

This example proves why playlist-based music recommendation is a difficult language task. Firstly, these songs are not necessarily hard rock. According to Spotify and other music platforms, these tracks lie in an alternative or independent music genre. Additionally, Elvis Costello's songs are from the 1970s, not from 2010. Even though the playlist title itself is descriptive, its contents do not fully match the semantics.

- Example #3: Artist playlist
- Playlist title: 2080
- Playlist length: 10 songs
- Songs include:
 - 1 Up - 2080
 - Extra Life - 2080
 - Heart Shaped Pixel - 2080
 - Maximized - 2080
 - The Master Key - 2080

While not the best use for a playlist, this example does follow the essential playlist "rules": an accurate playlist title and related songs. With all the songs from the same artist, these tracks are very closely related to each other.

- Example #4: Event playlist
- Playlist title: IOW 2012
- Playlist length: 35 songs
- Songs include:
 - Black - Pearl Jam
 - Free Fallin' - Tom Petty
 - The River - Bruce Springsteen

- Yellow Ledbetter - Pearl Jam
- Learning to Fly - Tom Petty And The Heartbreakers

This is another good example of a playlist title and usage. IOW 2012 represents the Isle of Wight Festival 2012, a British music festival. That year's festival was headlined by Tom Petty, Pearl Jam and Bruce Springsteen, which aligns with the playlist content. Although the playlist title matches an external event, the songs themselves may not always match semantically. Even if this example is arguably the best use case of a playlist, it is the most ambiguous and the hardest to predict. These examples offer various levels of unpredictability and evaluation difficulty, showing the underlying problems that exist when recommending tracks given a playlist title.

4 Methodology

This section describes the procedures used to evaluate naive, lexical, embedding and transformer-based recommendation models.

4.1 Naive Models

To confirm the task's plausibility, two naive models were implemented as baselines.

A random recommender system randomly retrieves ten songs from the entire collection and measures if the track was among the top 10 selections. As expected, this is a futile model used merely for a reasonableness check.

A popularity recommender system always retrieves the ten tracks with the most appearances in the collection. While this is still an ineffectual model, it does provide slightly better statistics due to probability.

Given that these models retrieve tracks and provide results, this task is considered an attainable one, leading to the development of more comprehensive models.

4.2 Non-naive Models

To truly evaluate the task's performance, five more sophisticated, singular-based models were implemented: two lexical, one embedding and two transformer-based systems. All non-naive models implement a retrieval process by identifying the top 30 most similar training playlists and retrieving the top 10 tracks for recommendation after aggregation and ranking.

Lexical Methods. These models consider playlists as documents of song tokens. Term Frequency-Inverse Document Frequency (TF-IDF) is a quantitative retrieval model that takes into account the frequency of a term in a document and the inverse frequency of a document in the collection. Each song within its document has its own weight depending on their number of

occurrences in the entire collection of playlists, with rare songs in specific playlists receiving higher weight values.

Best Matching 25 (BM25) is a probabilistic retrieval model that considers document length normalization in addition to term frequency. Similar to TF-IDF, playlists function as documents, where the document length normalization prevents long playlists from having high scores. This is particularly crucial for user-generated playlists where playlist size varies indiscriminately.

Embedding Method. Unlike TF-IDF and BM25 that consist of vectors of playlist tracks, Word2Vec uses sequences of tracks to predict neighboring tracks given a target song, and each playlist was represented as the mean of its song embeddings. This enables the model to capture semantic information about song patterns. Word2Vec embeddings capture semantic similarities between songs.

Transformer Methods. One of the main goals of this paper is to compare and contrast different models given their quality of recommendation and the computational efficiency. Transformer models have made recommendation systems a language task due to their large number of trainable parameters. The first transformer model was the pre-trained language model all-miniLM-L6-v2. After tokenizing the tracks in every playlist, the playlist is passed through an encoder and represented as the mean pooling of all outputs. Given its frozen nature, the model does not adapt to playlist-specific patterns and just functions as a general knowledge model.

Parameter-efficient fine-tuning (PEFT) is a quintessential approach to improve model efficiency without altering performance. The second transformer model applied PEFT by implementing low-rank adaptation (LoRA) to decrease the number of trainable parameters used for playlist adaptation. Additionally, simple contrastive learning (SimCSE) was used on a trainable playlist to learn more about semantically close representations.

4.3 Hybrid Models

Ensemble models have been created to compare the performance of more than one model simultaneously. The collaboration of different models can yield better results, especially by tuning weights. Four different hybrid models were implemented, with three of them having two different sets of weights.

Lexical Hybrid. This model was implemented by combining both TF-IDF and BM25 with a 50% weight split. Both models perform their rankings independently and each song's ranking is then averaged given the split. This hybrid model's main objective is to develop a superior text-based recommendation method.

Word2Vec Hybrid. A non-transformer hybrid model, this system combines lexical and embedding methods with two different sets of weights of 40% TF-IDF, 40% BM25 and 20% Word2Vec and another set of 30% TF-IDF, 30% BM25 and 40% Word2Vec. These weights were assigned to emphasize keyword matching or semantic information, respectively.

Transformer Hybrid. This model replaced Word2Vec with a pretrained transformer to compare static embeddings to a general knowledge model. Similarly to the Word2Vec hybrid, two sets of weights were used to highlight lexical and transformer models with a 25% TF-IDF, 25% BM25 and 50% transformer split and a 40% TF-IDF, 40% BM25, 20% transformer split, respectively.

PEFT Hybrid. The most ambitious model in this paper was the PEFT-based hybrid model, which consists of the text-based models with a PEFT transformer. This model uses a LoRA version of the pretrained transformer, allowing the decreased number of parameters to specialize in playlist patterns. The same two sets of weights from the transformer hybrid were used for this system to test whether fine-tuning returns better metrics over a pretrained transformer.

All hybrid models include lexical methods due to their interpretability and approach to keyword matching. However, patterns are not nearly as valuable without some context. Embedding and transformer methods capture this needed semantic information without matching track names with artists. This permits the recommendation systems to prioritize hit rate (finding songs by pattern matching) while still considering the semantic meaning between a track and its artist. This prioritization is explained through the lexical-heavy weights used in these hybrid models. Additionally, the hybrid splits were decided arbitrarily, and there was not an extensive search to find better splits.

4.4 Evaluation Metrics

Recommendation quality was evaluated using both unranked and ranked metrics used in information retrieval. Since the main task returns the top 10 tracks for each playlist title, the evaluation must consider both whether a relevant song was retrieved and its position.

For unranked evaluation, precision, recall and F1 were used to assess the retrieval of a track. Precision@10 measures the proportion of relevant songs among the top 10 recommendations, emphasizing recommendation accuracy. On the other hand, Recall@10 measures the proportion of relevant songs retrieved among all relevant songs in the target playlist, emphasizing coverage. By calculating the harmonic mean of Precision@10 and Recall@10, F1@10 is included to balance accuracy and coverage.

Another unranked metric used in this paper is HitRate@10. Hit Rate is a binary evaluation measure that checks whether at least one relevant song was retrieved within the top 10

recommendations. This is useful for identifying if the model can produce at least one valid recommendation, serving as a benchmark of model success.

For ranked evaluation, MRR, MAP and NDCG were used to grade the quality of the track retrieval. Mean Reciprocal Rank (MRR) measures how early the first relevant song appears. This metric benefits early retrieval. Mean Average Precision (MAP) summarizes precision at all relevant positions, evaluating both precision and ranking position. Additionally, Normalized Discounted Cumulative Gain (NDCG@10) considers both relevance and position, giving higher weight to tracks that are both relevant and appear early.

Because playlist titles are often ambiguous, multiple metrics are required to provide complementary perspectives on recommender performance. In addition to recommendation quality, runtime differences between models were recorded to compare efficiency.

As for the model evaluations, this work assumes a song is relevant if and only if it appears in the correct playlist. While this is a decent assumption to make, this also presumes that any song not on the original playlist is not relevant, which is not accurate in real-world recommendation since a playlist is not necessarily a ground truth for a certain genre, mood, event or artist. This assumption undermines recommendation quality.

5 Experiments

Experimental settings and hyperparameters are explained in this section.

5.1 Experimental Settings

To improve the overall training sessions, CUDA was used. Additionally, random seeds were set for consistency (seed = 16). As mentioned, Spotify user playlists were used with a 90/10 train test split, using a sample of 200 test playlists. A subset of 200 playlists was used due to computational constraints, and larger testing subsets might provide slightly different results.

5.2 Hyperparameters

TF-IDF. Scikit learn has a tf-idf vectorizer that converts documents into TF-IDF matrices ready for computation. Some hyperparameters were tuned to develop better results. For example, the addition of trigrams along with unigrams and bigrams produced better results, indicating that three words is a decent number for track name and artist name. Additionally, 5000 features was a sweet spot between quality and efficiency, as 10000 features proved to be too expensive.

As for document frequency, I figured that a minimum and maximum between 0.8 and 2 was the best combination of rare and common terms. A combination of song and artist must be unique enough to avoid too many repetitions in a playlist, but not so rare it skews its TF-IDF rating to

the point of ignoring the other tracks in the playlist. Sublinear term frequency was used to improve scaling and avoid spikes.

BM25. BM25 Okapi has its own class with its two distinct parameters: k_1 and b controlling term frequency saturation and document term normalization, respectively. As per the documentation, the default values of these hyperparameters are $k_1=1.5$ and $b=0.75$. Given the high metrics these values yielded, no additional tuning has been carried out. However, this could be an area of improvement.

Word2Vec. Gensim was used. Various hyperparameters were tuned to improve performance, but most importantly, reduce computational cost. Firstly, the original vector size of 200 songs was very expensive, and while performance took a slight hit (still less than lexical based models), a vector size of 50 had a proper quality-efficiency value. Other changes made to improve training time that did not affect performance were window size from 10 to 5 and epochs from 20 to 10.

To optimize performance, $\text{count}=1$ was set to include all songs, improving the hit rate. Additionally, skip-gram was implemented. This helps represent rare words, which is key for retrieval.

Frozen Transformer. The frozen transformer model used in this work was all-miniLM-L6-v2. This compact model consists of 22.7 million trainable parameters. It was trained on around 1 billion training tuples and maps sentences to 384 dimensional vector space. As for hyperparameters, only batch size and max length were used in this model. A batch size of 64 provided better performance over 32, and max length of 128 surpassed 256. To normalize the embeddings, the default values of exponents and dimensions were 2 and 1, respectively.

PEFT Transformer. Parameter-efficient fine-tuning is the main goal of this work. Low-Rank Adaptation (LoRA) was used to reduce computational cost. LoRA was computed using LoraConfig with an attention dimension (rank) of $r=8$ and an alpha of $\text{lora_alpha}=16$, which are the standard values with a scaling factor of 2. Additionally, the dropout rate was initialized to 0.1 and the adapter was applied to all three attention vectors: query, key, value. So, with an original transformer with 22.7 million parameters and a vector space of 384 dimensions (384×384 weight matrices for query, key and vector), the matrix dimensions were reduced to 384×8 , greatly reducing the total number of trainable parameters.

With the average model taking around 35 minutes to train, temperature, dropout, learning rate, batch size, max length and number of epochs were all tuned hyperparameters. Fine-tuning started with a learning rate of 0.0002, a batch size of 64, a max length of 128 and one epoch. For readability purposes, a small legend used to abbreviate the hyperparameters tested on the transformer was created. Below are the results of all fine tuning. The best numbers are in **bold**:

- Temperature (T)
- Dropout (D)
- Learning Rate (LR)
- Batch Size (BS)
- Max Length (ML)
- Epochs (E)

Temperature and Dropout Check								
Fixed		LR = 0.0002, BS = 64, ML = 128, E = 1						
T	D	Precision@K	Recall@K	F1@K	HitRate	MRR	MAP	NDCG
0.05	0.1	0.0700	0.2179	0.0892	0.4550	0.2924	0.1499	0.3300
0.05	0.2	0.0715	0.2046	0.0894	0.4550	0.2893	0.1381	0.3288
0.1	0.1	0.0775	0.2326	0.0990	0.5100	0.2659	0.1195	0.3183
0.1	0.2	0.0765	0.2310	0.0978	0.4850	0.2646	0.1209	0.3097
0.2	0.1	0.0730	0.2120	0.0910	0.4800	0.2316	0.1003	0.2892
0.2	0.2	0.0710	0.2097	0.0892	0.4450	0.2198	0.1039	0.2731

Table 2: Metrics on different temperature and dropout values. While a temperature of 0.05 resulted in the better ranked metrics, a temperature of 0.1 and a dropout of 0.1 were chosen as these values provided the best unranked metrics.

Learning Rate and Batch Size Check								
Fixed		T = 0.1, D = 0.1, ML = 128, E = 1						
LR	BS	Precision@K	Recall@K	F1@K	HitRate	MRR	MAP	NDCG
0.0001	32	0.0740	0.2233	0.0952	0.4700	0.2697	0.1237	0.3124
0.0001	64	0.0795	0.2238	0.0995	0.5150	0.2704	0.1151	0.3224
0.0002	32	0.0800	0.2359	0.1013	0.5350	0.2817	0.1223	0.3365
0.0002	64	0.0775	0.2323	0.0989	0.4900	0.2673	0.1228	0.3143
0.0005	32	0.0830	0.2405	0.1037	0.4950	0.2986	0.1393	0.3389
0.0005	64	0.0830	0.2351	0.1039	0.5300	0.2843	0.1254	0.3400

Table 3: Metrics on different learning rates and batch size values. A higher learning rate of 0.0005 combined with a batch size of 64 provided the best precision and NDCG while having a top three performance in hit rate, MRR and MAP.

Max Length Check								
Fixed	T = 0.1, D = 0.1, LR = 0.0005, BS = 64, E = 1							
ML	Precision@K	Recall@K	F1@K	HitRate	MRR	MAP	NDCG	Time
128	0.0770	0.2140	0.0961	0.5000	0.2841	0.1263	0.3332	~35:00
256	0.0835	0.2320	0.1032	0.4950	0.2726	0.1236	0.3169	~45:00

Table 4: Metrics on different maximum lengths. A max length of 128 makes sense because, on average, a playlist has 20 songs, and ~6-7 tokens per song feels like a decent amount for a “Song Artist” string. Additionally, it had a slightly more efficient run.

Epochs Check								
Fixed	T = 0.1, D = 0.1, LR = 0.0005, BS = 64, ML = 128							
E	Precision@K	Recall@K	F1@K	HitRate	MRR	MAP	NDCG	Time
1	0.0795	0.2313	0.1004	0.4900	0.2914	0.1374	0.3309	~35:00
2	0.0805	0.2244	0.1011	0.4850	0.2745	0.1302	0.3174	~65:00
3	0.0815	0.2296	0.1013	0.5100	0.2953	0.1359	0.3407	~95:00

Table 5: Metrics on a different number of epochs. Three epochs provided the best values while being the most computational expensive in a slightly nonlinear way. Only one epoch could have resulted in the best quality metrics given its efficiency; however, three epochs gave the best metrics in most statistics.

So, with a temperature of 0.1, a dropout rate of 0.1, a learning rate of 0.0005, a batch size of 64, a maximum length of 128 and three epochs, we have optimized our LoRA-based transformer.

5.3 Efficiency Measurements

Training time was measured using the runtime of each function from call to completion. This does not include the runtime it took to load and prepare the data. All models were trained on the same system with the same GPU (CUDA) and the same seed for reproducibility. The timing of each model includes general evaluation methods like cosine similarity as well as model-specific implementations, such as the matrix creation for TF-IDF, the tokenization and contrastive loss of transformer-based models and different weighting factors for hybrid-based recommenders.

6 Results

This section presents the metrics from all the models. The best numbers are in **bold**, second best underlined, third best *italicized*.

6.1 Model Performance

Single Recommenders								
Model	Precision@K	Recall@K	F1@K	HitRate	MRR	MAP	NDCG	Time
Random	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	00:25
Popular	0.0005	0.0017	0.0008	0.0050	0.0006	0.0002	0.0016	00:25
TF-IDF	<u>0.0700</u>	0.1912	0.0883	<u>0.4550</u>	<i>0.2306</i>	0.0913	<i>0.2771</i>	<i>15:00</i>
BM25	0.0670	0.2301	<u>0.0915</u>	<i>0.4400</i>	0.1606	0.0788	0.2250	<i>15:00</i>
W2V	0.0655	0.1803	0.0792	0.4000	0.2032	<i>0.1012</i>	0.2487	18:07
Mini-LM	<i>0.0685</i>	<i>0.2152</i>	<i>0.0888</i>	0.4350	<u>0.2369</u>	<u>0.1153</u>	<u>0.2836</u>	<u>9:41</u>
PEFT	0.0815	<u>0.2296</u>	0.1013	0.5100	0.2953	0.1359	0.3407	95:00

Table 6: Metrics on all single (non-hybrid) recommenders.

For K=10, the PEFT model achieved the best metrics while being the most computationally expensive by a wide margin. The lexical single models (TF-IDF, BM25) had decent unranked metrics. The frozen transformer (Mini-LM) had the second best ranked metrics, surpassing all non-PEFT models on MRR, MAP and NDCG while being the least computationally expensive non-naive model. Word2Vec (W2V) performed among the top three in MAP only.

Hybrid Recommenders					
Model	Split	Precision@K	Recall@K	F1@K	HitRate
TF-IDF/BM25	50/50	<i>0.1025</i>	0.2940	0.1294	<u>0.6150</u>
TF-IDF/BM25/W2V	30/30/40	0.0885	0.2607	0.1113	0.5650
TF-IDF/BM25/W2V	40/40/20	0.0990	0.2872	0.1250	<i>0.6050</i>
TF-IDF/BM25/LM	25/25/50	0.0975	0.2782	0.1215	0.5750
TF-IDF/BM25/LM	40/40/20	0.1050	<u>0.2948</u>	0.1313	0.6250
TF-IDF/BM25/PEFT	25/25/50	<u>0.1040</u>	0.2975	<i>0.1301</i>	0.6000
TF-IDF/BM25/PEFT	40/40/20	0.1050	<i>0.2945</i>	<u>0.1311</u>	0.6250

Table 7: Unranked metrics on all hybrid recommenders.

For the hybrid models, the transformer-based ones outperformed the other hybrid models in unranked metrics. The frozen transformer hybrid with a 40/40/20 split had the best precision, F1 score and hit rate along with the second best recall. The parameter-efficient fine-tuned hybrid models (both splits) rounded up most of the unranked metrics top three. The lexical hybrid had a top three precision and hit rate, and the Word2Vec hybrid had the third best hit rate.

Hybrid Recommenders					
Model	Split	MRR	MAP	NDCG	Time
TF-IDF/BM25	50/50	<i>0.3850</i>	0.1922	<i>0.4374</i>	27:55
TF-IDF/BM25/W2V	30/30/40	0.3227	0.1599	0.3806	46:47
TF-IDF/BM25/W2V	40/40/20	0.3702	0.1841	0.4241	49:06
TF-IDF/BM25/LM	25/25/50	0.3439	0.1689	0.3964	<i>41:38</i>
TF-IDF/BM25/LM	40/40/20	<u>0.3856</u>	<i>0.1894</i>	<u>0.4393</u>	<u>40:54</u>
TF-IDF/BM25/PEFT	25/25/50	0.3818	0.1764	0.4105	106:47
TF-IDF/BM25/PEFT	40/40/20	0.3863	<u>0.1919</u>	0.4419	105:00

Table 7: Ranked metrics and training time on all hybrid recommenders. The best numbers are in **bold**, second best underlined, third best *italicized*.

The lexical hybrid (TF-IDF/BM25) was among the top three in all ranked metrics (with the best MAP) while being the least computationally expensive by being the only model to yield results in under thirty minutes. The frozen transformer hybrid (TF-IDF/BM25/LM) was tied with the best hit rate at 62.5% and the second best MRR and NDCG while being the second quickest model. However, the model with the best performance was the parameter-efficient fine-tuned hybrid (TF-IDF/BM25/PEFT) that was tied with the best hit rate while also having the best MRR and NDCG and the second best MAP. Nevertheless, it had a significantly higher cost at 105 minutes, which was more than double the next non-PEFT model.

Another finding from the hybrid recommenders is the overexposure of lexical models. All top three performances on every metric involved a high usage of TF-IDF and BM25. In fact, the changed models involving more weight on these lexical models all outperformed the original hybrids, demonstrating a high overall performance of these keyword-based models.

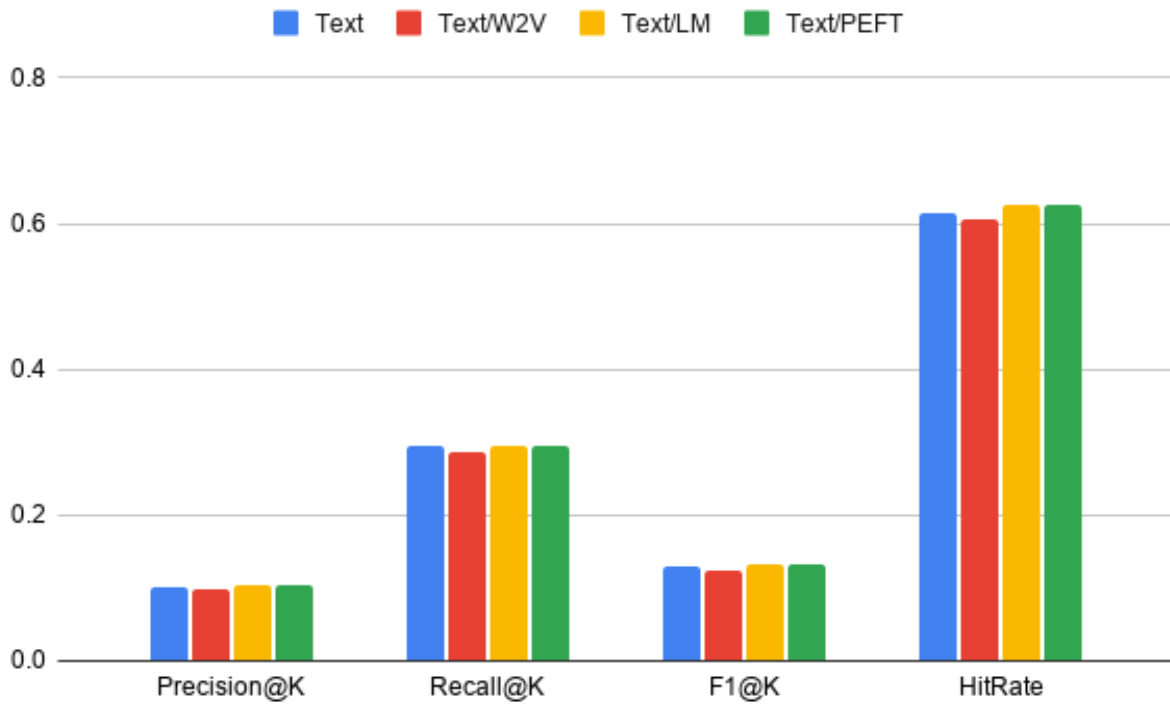


Figure 1: Unranked metrics for K=10 for hybrid recommenders (40/40/20 split for three-model recommenders).

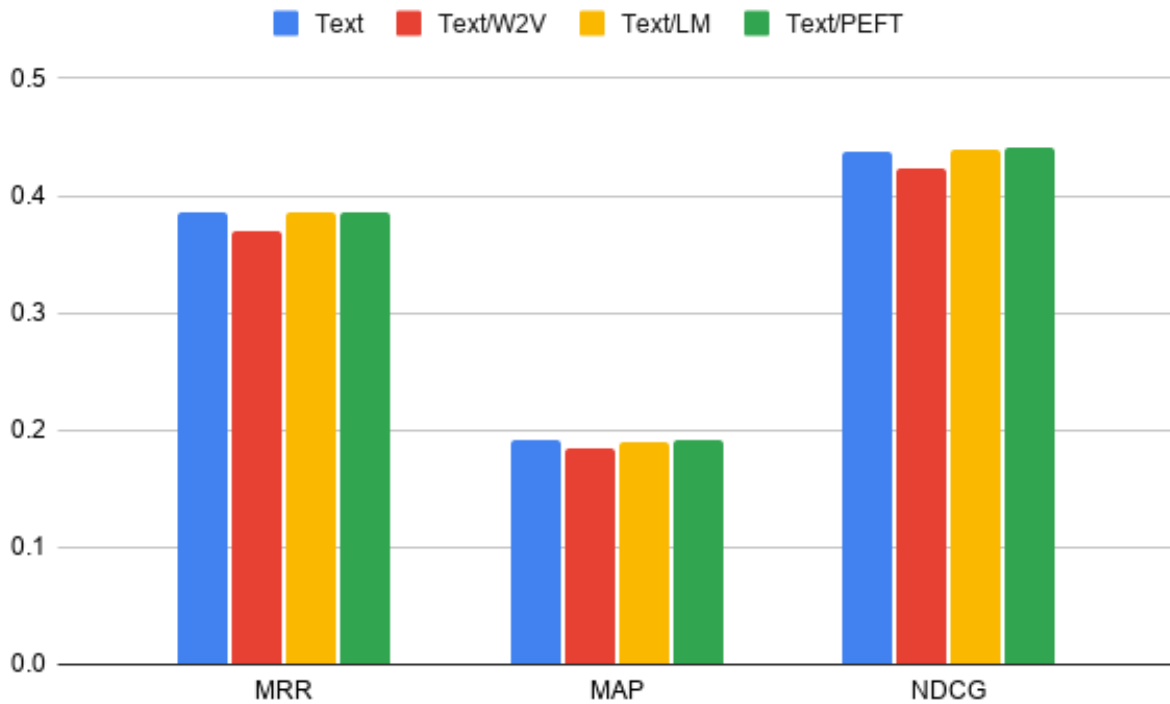


Figure 2: Ranked metrics for hybrid recommenders (40/40/20 split for three-model recommenders).

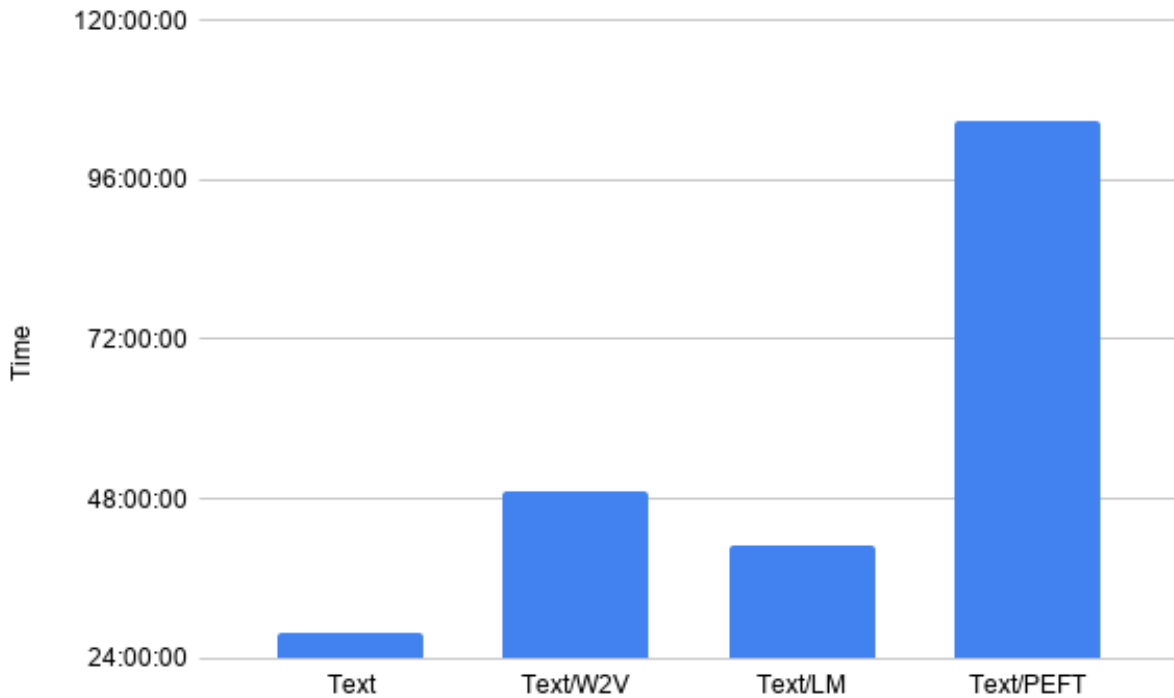


Figure 3: Computational cost for hybrid recommenders (40/40/20 split for three-model recommenders).

6.2 Efficiency Analysis

Non-hybrid models have the fastest training time due to only preparing the data for one process. More specifically, the frozen transformer was the only non-naive model to return results in under ten minutes. For a hit rate of 43.5%, it is a decently good quick model. On the other hand, Word2Vec embeddings produced the least favorable metrics with a decently short training time. Both lexical single models had similar metrics.

More models in a hybrid ensemble mean more computational cost. The text-based model was the only model composed of two single models. It also produced the best efficiency, with 61.5% hit rate in less than 30 minutes. However, the model with arguably the best results is the text-based model combined with a fine-tuned transformer (80/20 split). Nevertheless, this comes at around four times more training time for marginal gains in these statistics. In practice, the text-based model is the most efficient model.

7 Discussion

This section provides analysis about the results of the models.

7.1 Interpretation of Results

The main takeaway from these experiments is that hybrid models perform better than single models due to their combination of different strengths. All hybrid models included lexical systems due to the powerful tool of keyword matching. As for all the models, the hybrid model with a parameter-efficient fine-tuned transformer had the best performance with a hit rate of 62.5% and a NDCG of 0.4419. Following closely is the hybrid model with a frozen transformer, which had an identical hit rate with lower ranked metrics.

The most surprising result of these experiments was the strong performance of the lexical hybrid model. TF-IDF and BM25 combined made up the third best model from the list, with the highest mean average precision of any model.

The most disappointing result of these experiments was the lackluster performance of Word2Vec embeddings. The single model had a 10% lower hit rate than the lexical counterparts. Additionally, any hybrid with Word2Vec in it performed worse than the rest.

A single model consisting of a PEFT model did provide the best performance compared to the rest of the single models, proving that transformers do excel in semantics and pattern recognition by being the only single model with a hit rate of over 50% along with being the best single model in all metrics except recall. However, this comes at a staggering cost that sometimes can be mitigated with a combination of other models.

Proving that classical information retrieval analysis can still provide valuable insights over computationally expensive models can aid streaming services and other recommendation-based applications by saving resources. However, this work solely focuses on the recommendation aspect of this information retrieval task. It does not provide any conversation or reasoning behind those decisions, making the experience not user-driven.

7.2 Quality-Efficiency Tradeoff

The best performing models are not necessarily the best models to implement. There is a point of diminishing returns, especially when considering the amount of computational resources used to improve metrics by the decimal.

For example, the best performing model was trained and tested for 105 minutes. Additionally, other embedding-based hybrid models took over 40 minutes to train and test. On the other hand, the lexical-based hybrid model performed the best relative to their training cost. It developed into the third best model with less than 30 minutes of training time.

As for single models, the most efficient model was the frozen pretrained transformer. Being the only non-naive model to produce results in under 10 minutes, it was among the best three models, beating both TF-IDF and BM25 on all three ranked metrics by a decent margin. TF-IDF and BM25 outperformed the pretrained transformer model on hit rate with approximately 15 minutes of training time for each.

8 Conclusion

This work demonstrates a clear initial foundation for playlist retrieval recommendation. The comparison of lexical, embedding and transformer models provides systemic results. Another important conclusion from this work is that transformers and parameter-efficient fine-tuning are not always the best solution for a retrieval problem. Although the hybrid PEFT model resulted in the best metrics, its computational cost was not worth the marginal gains. This also confirms the importance of lexical-based models as an appropriate benchmark for more sophisticated retrieval systems.

Limitations

While many limitations were identified while performing the experiments, three were primarily recognized:

A primary limitation of this work is using training time as the only efficiency metric. While training time represents a tangible measure of computational cost, this number can greatly vary among technologies. Other efficiency metrics that could supplement training cost include hardware characteristics, GPU usage and other memory consumption. These measures could have provided a more detailed quality-efficiency analysis.

This work uses all-MiniLM-L6-v2, a relatively lightweight transformer with 22.7M parameters, as the transformer for all the models. While the average BERT-based transformer has at least 110M parameters, the decision to use all-MiniLM-L6-v2 instead lied on pure cost-benefit analysis and computational power, as using a larger transformer would require more GPU usage and training time. This limitation halts a proper comparison with state-of-the-art large language models.

A final limitation states low-rank adaptation as the only PEFT method used in this work. While LoRA is the main PEFT method used to reduce the number of trainable parameters, others such as prefix tuning or quantized LoRA could have been tested to compare parameter efficiency.

References

- [1] Pichl, M., Zangerle, E., Specht, G. (2015). "Towards a Context-Aware Music Recommendation Approach: What is Hidden in the Playlist Name?" *15th IEEE International Conference on Data Mining Workshops (ICDM 2015)*, pp. 1360-1365.
- [2] Pichl, M., Zangerle, E. (2021). "User Models for Multi-Context-Aware Music Recommendation." *Multimedia Tools and Applications, Volume 80, Issue 15*, pp. 22509 - 22531.
- [3] Pichl, M., Zangerle, E., Specht, G. (2016). "Understanding Playlist Creation on Music Streaming Platforms." *2016 IEEE International Symposium on Multimedia (ISM)*, pp. 475-480.
- [4] Ravaut, M., Zhang, H., Xu, L., Sun, A., & Liu, Y. (2024). "Parameter-efficient Conversational Recommender System as a Language Processing Task." *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2024), Volume 1: Long Papers*, pp. 152–165.
- [5] Sarode, K., & Javaji, S. R. (2023). "Multi-BERT for Embeddings for Recommendation System." arXiv:2308.13050.
- [6] Hu, E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. & Chen, W. (2021). "LoRA: Low-Rank Adaptation of Large Language Models." arXiv:2106.09685v2.
- [7] Gao, T., Yao, X. & Chen, D. (2022). "SimCSE: Simple Contrastive Learning of Sentence Embeddings." arXiv:2104.08821.